Reflective SQL Debugging Agent

CSE P 590 Project Writeup

Sam Millar (ID# 2550529) (shmillar@uw.edu)
Joannier Pinales Hevia (ID# 2340661) (jopinale@uw.edu)
Ann Baturytski (ID# 1902090) (abatur@uw.edu)

Abstract

We built a reflective SQL debugging agent on Llama-3.1-8B-Instruct served by vLLM on a Cloud TPU v5litepod-4, and measured it at both the application level and the engine level across 100 broken Spider queries. Reflection works: success rises monotonically from 0% with one round to 33–40% with five. It also costs: prompts grow 2.3× across rounds, and per-round latency grows with them. Prefix caching absorbs much of that growth, cutting mean time-to-first-token by 48%, our firmest quantitative finding. Chunked prefill showed no effect at concurrency 2, a null result we attribute to experimental scale rather than to the optimization. No engine knob moved accuracy: easy-tier success sat at exactly 52.9% in every condition and the hard tier stayed under 25%, placing the ceiling on model capability rather than infrastructure.

1. Introduction

Large language models are increasingly deployed as autonomous agents that loop over tool calls, observe feedback, and revise their outputs across multiple steps. The reflection pattern (generate, execute, observe, revise) provides structured execution feedback and can substitute for the model's unverifiable internal reasoning [1, 2]. However, reflection comes with a systems cost: each round appends the previous attempt and its result to the prompt, so context length grows monotonically with the number of iterations. On a batch inference engine like vLLM [3], this means prefill work compounds across rounds, and longer prompts from later steps of reflection can delay shorter requests from other tasks in the queue.

SQL Debugging is a natural fit for this pattern. Execution feedback is exact: a syntax error names the offending token, a semantic error returns wrong rows, a slow query exposes its bottleneck in the execution plan. Every task carries a ground-truth query for reference (though, as we note in the methodology, our deployment could not verify result equivalence at runtime). Difficulty is tunable, making it possible to ask whether a given model or infrastructure configuration saturates at a particular tier.

We built a reflective SQL debugging agent on top of Llama-3.1-8B-Instruct [4] served via vLLM on a Cloud TPU v5litepod-4, and evaluated it on 100 broken queries derived from the Spider benchmark [5] spanning three perturbation tiers. Our experiment sweeps two variables: reflection budget (up to 5 rounds) and vLLM engine configuration (prefix caching [6] & chunked prefill [7], 4 presets). This lets us ask two questions at once. At the application level: how much

does iterative reflection actually help, and does prompt verbosity matter? At the engine level: can prefix caching contain the prefill cost growth that the reflection pattern necessarily induces, and does chunked prefill reduce tail latency under concurrent load?

2. System Design

The Reflective SQL Debugging Agent is a stateful Python process built around the reflection loop. At each step it gathers a single prompt (system prompt, database schema, original broken query, and the full ordered history of prior attempts with their tool results) and submits it to the vLLM endpoint. Output is parsed for a single JSON-formatted tool call and dispatched to one of two tools.

The SQL executor submits a candidate query to the local PostgreSQL instance and returns either PostgreSQL's standard error message or the first several result rows on success. The query plan inspector then runs EXPLAIN ANALYZE on the candidate query and returns the full execution plan. This tool is the main signal for harder level tasks where the query produces correct rows, but the execution strategy is the target of the fix.

A task terminates when the model calls final_answer(), signaling it considers the query debugged, or when the agent hits its reflection budget. Because no row data is loaded into PostgreSQL, the agent cannot verify correctness at runtime by comparing output against a ground-truth result set; success is the agent's own final_answer() report, with the harness additionally recording a post-hoc structural match against the ground-truth query.

Prompt verbosity is one of our two sweep variables, and it's handled where the agent builds the prompt each round. In full mode, the agent pastes the entire tool result into the history. In compact mode, any tool longer than 600 characters gets cut off, with a note saying how long it originally was. This changes how fast the prompt grows each round, and lets us check whether giving the model less detail in its feedback hurts its ability to fix the query, separately from any engine settings. This also helps us to shorten the prompts so we can optimize the reflection budget.

3. Infrastructure

We served Llama-3.1-8B-Instruct with vLLM on a Cloud TPU v5litepod-4 (4 chips, 16 GB HBM each, 2x2 topology) in GCP zone us-south1-a. The model was sharded across all four chips with tensor parallelism (TP=4) at bfloat16, with a max context length of 8,192 tokens. After weights were loaded, the engine reported 13.5 GiB used out of a 58 GiB usable HBM. That left a KV cache of ~363,776 tokens, which is about 44 concurrent full-context requests, so KV cache capacity was never a constraint from our benchmarks of 2.

We ran the `vllm/vllm-tpu: nightly` Docker image instead of installing vLLM directly on the TPU VM. We tried the pip route first, and it failed three different ways. The current vLLM release required a Torch version newer than what torch_xla's TPU wheels support. Pinning an older

vLLM didn't work either, as it expected paged-attention XLA ops that only exist in later torch_xla versions. And the transformers version in between used float8 dtypes that the older torch doesn't have. The container ships a pre-validated combination of torch, torch_xla, libtpu, and JAX, so the host VM only needed Docker and PostgreSQL. This was a practical finding on its own: on TPU, the deployment unit for vLLM is the container, not the Python package.

Model choice was also constrained by the serving stack in GCP. The vllm-tpu image routes models through `tpu_inference`, which has JAX-native implementations for a fixed list of architectures (Llama, Qwen, Gemma, DeepSeek, and a few others). Mistral-7B, one of our proposal's candidates, is not on that list. It fell back to a PyTorch path that segfaulted during engine initialization. Qwen2.5-7B failed differently: the loader assumes a multimodal-style config with a `text_config` field that Qwen2's text-only config doesn't have. Llama-3.1-8B-Instruct uses the JAX-native `LlamaForCausalLM` path and worked without modification. The attention backend auto-selected flash_attn, a Pallas flash-attention kernel [8].

The two engine knobs for our study are prefix caching (RadixAttention [6]) and chunked prefill [7], which we passed as startup flags. Their on/off combinations gave the four engine presets used in the sweep, and each preset was held fixed for an entire benchmark. Switching presets required a container restart, which took about three minutes with weights cached on the local disk. First boot was slower: weight load took ~6 minutes, and XLA precompilation of the prefill, decode, sampling, and structured-decoding graphs added another ~100 seconds.

We exposed an OpenAI-compatible endpoint on port 8000, reachable from client machines over an SSH tunnel via port forwarding. PostgreSQL ran on the same VM as the SQL execution target. The agent's SQL executor and the inference engine shared a host, so round-trip latency was dominated by inference rather than network hops. Engine telemetry came from vLLM's Prometheus `/metrics` endpoint, which we snapshotted before and after each sweep to derive TTFT and end-to-end latency from histogram sum/count pairs, plus prefix cache hit rate and KV cache utilization. We ran a single smoke-test request (96 completion tokens), which completed in 0.73 s end-to-end with a 0.16 s time-to-first-token. We used this to confirm the endpoint was interactive-grade before benchmarking. The VM cost roughly $115/day while running, so we kept it stopped except during active sweeps.

4. Benchmark & Methodology

We constructed a dataset of 100 broken SQL queries from the Spider dataset [5], double the ~50 tasks the proposal planned, so that each difficulty tier keeps ~33 tasks. Each task records a natural language question, a database schema, a broken query, and a ground truth query. We calculated query difficulty based on the SQL operators within the query. Easy queries contain syntax errors, medium queries contain semantic errors, and hard queries contain performance anti-patterns. Success is reported by the agent when it calls final_answer() within the reflection budget, and every success rate in this paper is that self-reported metric; the harness separately records a stricter post-hoc structural match against the ground-truth query. Since there is no row data loaded into Postgres, the agent cannot verify correctness by matching expected results,

and self-reported success can overcount: the agent may declare victory on a query that runs cleanly but is still semantically wrong. The application-level success numbers should therefore be read as an upper bound. This is the one place our setup falls short of the proposal's plan to verify against ground-truth results automatically.

We swept across two independent variables (prompt verbosity and engine configuration) to create 8 total conditions with 100 tasks, with up to 5 reflection rounds and concurrency of 2 parallel tasks. In full prompt verbosity, the agent receives the complete tool result at each round. In compact prompt verbosity, tool results are truncated to a brief summary. This variable controls how fast context grows per round and is implemented in the agent's prompt assembly. We use prefix caching (RadixAttention on/off) with chunked prefill (on/off), producing four combinations:

- prefix_on & chunked_on
- prefix_on & chunked_off
- prefix_off & chunked_on
- prefix_off & chunked_off

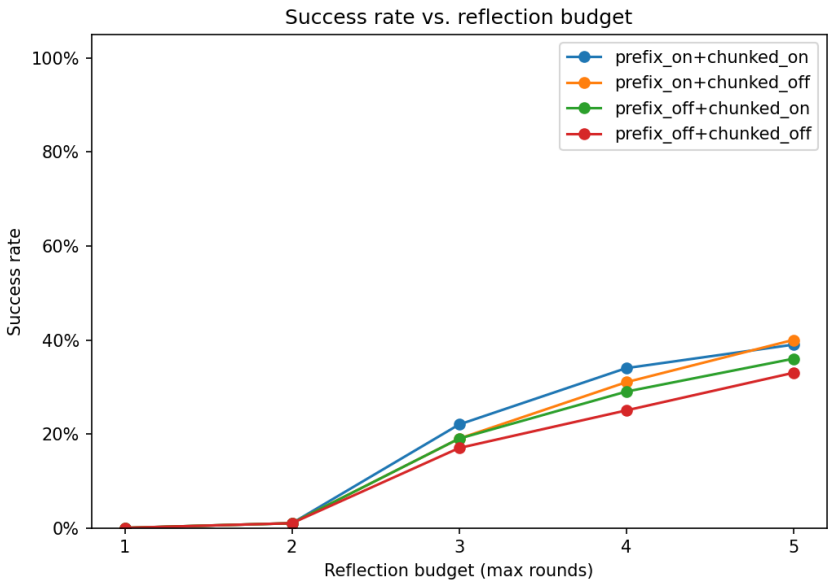
These are passed as vLLM startup flags and held fixed for the duration of each sweep run.

Application-level metrics are recorded per task in JSONL output by the harness. Engine-level metrics are collected via vLLM's Prometheus metrics endpoint snapshots immediately before and after each sweep run. We derive mean time-to-first-token (TTFT) and mean end-to-end request latency from the histogram sum/count pairs, along with GPU prefix cache hit rate and KV cache utilization. We compared conditions using Mann-Whitney U for continuous metrics (tokens, latency, rounds) and Fisher's exact test for success rates, with rank-biserial correlation as an effect size estimate.

Two more deviations from the proposal are worth flagging up front. We capped the reflection budget at 5 rounds rather than sweeping to 10: the two-variable sweep already required eight full benchmark runs on a VM costing ~$115/day, and the budget curve below can be read at every cutoff up to 5 from the same runs. We also do not report decode throughput or queue wait time from the proposal’s instrumentation list. At concurrency 2 the queue depth stayed at zero, so queue wait is trivially uninformative, and decode lengths were short (~20-30 completion tokens per round) and roughly constant, so per-round latency differences are driven by prefill. Finally, the planned non-reflective smolagents baseline was cut for time; the budget-1 condition plays that role, since a budget of 1 is exactly single-shot generation with no execution feedback.

5. Results

5.1. Reflection Budget



The above plot shows the success rate as a function of the maximum reflection budget allowed, aggregated across all engine configurations. With one reflection round, no task is solved; part of this is mechanical, since at budget 1 the agent must answer before seeing any execution feedback, making it exactly the single-shot, non-reflective baseline. At two rounds, fewer than 2% of tasks succeed. The rate rises sharply to 17–22% at three rounds and continues growing through round 5, where it reaches 33–40% depending on engine configuration.

Two properties of this curve are notable. First, the single-shot baseline solves nothing, and near-nothing arrives before round 3: the model needs to observe execution feedback, usually more than once, before it produces a correct query. This directly validates the iterative reflection pattern we chose for this project. Second, the slope from round 4 to round 5 remains positive across all conditions, suggesting that continued increasing of the budget would continue to increase the success rate.

5.2. Prefix Caching

Prefix caching behaved almost exactly as hypothesized.

Engine config	Mean TTFT	Mean e2e latency
prefix_on + chunked_on	**26.8 ms**	546 ms
prefix_on + chunked_off	**27.3 ms**	556 ms
prefix_off + chunked_on	51.7 ms	592 ms
prefix_off + chunked_off	51.9 ms	595 ms

Enabling prefix caching cut mean time-to-first-token by ~48%, from ~52 ms to ~27 ms. Our proposal predicted a 30-60% reduction, and this landed in the middle of that range. For a given task, every reflection round shares an identical leading context: the system prompt, the database schema, and the original broken query. RadixAttention serves that prefix from cache, so only the new content of each round (the previous attempt and its tool result) gets prefilled fresh.

Mean prompt length grew from 547 tokens in round 1 to 1,242 tokens by round 5 (2.3x, below our 3-5x prediction, Spider schemas and error messages are compact). Per-round latency tracked this growth, but more steeply without caching. Round 5 took 1.8x round-1 latency with prefix caching on (0.475 s to 0.849 s) versus 2.0x with it off (0.483 s to 0.954 s). The cached prefix is a larger share of the prompt exactly when the prompt is longest, which is where the savings matter most.

At the application level, the benefit showed up but was not statistically significant. Median task latency was 3.05 s with prefix caching versus 3.25-3.38 s without (Mann-Whitney p=0.08-0.09), about 0.3 s per task, or ~30 s across the 100-task benchmark. Success rate, tokens consumed, and rounds-to-success were unchanged (all pairwise $p > 0.37$). That is the expected result: prefix caching changes how fast each round is served, not what the model generates. The engine-level numbers are direct measurements rather than noisy task outcomes, so we consider the 48% TTFT reduction the firmest quantitative finding of our experiment.

5.3. Chunked Prefill

Our proposal predicted that chunked prefill would reduce tail latency under concurrent load by preventing long late-round prefills from blocking shorter requests (head-of-line blocking). However, we did not observe this:

Engine config	p50	p75	p95	p99
prefix_on + chunked_on	3.05 s	3.69 s	5.06 s	6.46 s
prefix_on + chunked_off	3.07 s	3.99 s	4.99 s	6.28 s
prefix_off + chunked_on	3.25 s	4.22 s	5.14 s	6.36 s
prefix_off + chunked_off	3.38 s	4.03 s	5.33 s	6.57 s

Within each prefix-caching group, the chunked-on and chunked-off distributions differed by less than 0.2 s at p95/p99, and the direction was inconsistent. Mean TTFT was likewise indistinguishable (26.8 vs 27.3 ms with prefix on, 51.7 vs 51.9 ms with prefix off).

We attribute this to the experimental scale rather than to the optimization itself. The hypothesis was untestable at concurrency 2, so a good follow-up would be to test this with higher concurrency. Chunked prefill helps by interleaving decode steps of running requests with slices

of a long incoming prefill [7], building on the iteration-level scheduling introduced by Orca [9]. That requires a queue deep enough for long and short requests to actually contend. With only two concurrent tasks, and per-round prompts that maxed out around 1,200-2,000 tokens (at or below our 2,048-token `max_num_batched_tokens` budget), a round-5 prefill almost never blocked a round-1 request from the other task. KV cache utilization stayed near zero throughout the runs, which confirms the engine was never under load.

Two conditions would make the predicted effect visible: higher concurrency (8-16 parallel tasks, where queueing is routine) and longer prompts that exceed the batched-token budget and force multi-chunk prefills. Both are within reach of this setup. The v5litepod-4's 44x KV-cache concurrency headroom means the engine itself would not be the bottleneck.

5.4. Agent Success by Difficulty

Success rates differ sharply across the three difficulty tiers and are largely insensitive to engine configuration (table below). The easy tier (syntax errors) converges to exactly 52.9% in every condition tested: the value is identical across all four engine presets and both verbosity levels. This invariance suggests a model capability ceiling rather than an infrastructure or information-delivery effect. The 47% of easy tasks that fail appear to involve error messages the model cannot convert to a valid fix within five rounds, not tasks that would succeed with a faster engine or longer tool output.

Tier	N	Best engine config	Worst engine config	Full verbosity	Compact verbosity
Easy	34	52.9%	52.9%	52.9%	52.9%
Medium	33	51.5%	27.3%	39.4%	33.3%
Hard	33	24.2%	15.2%	24.2%	18.2%

Medium tasks (semantic errors: wrong join types, swapped aggregations, inverted boolean logic) show the widest spread across conditions, 27-52% by engine config, and are the tier most sensitive to the variables under sweep. The extremes are not even the same preset: medium peaks at 51.5% under prefix_on + chunked_off and bottoms out at 27.3% under prefix_off + chunked_off, while hard peaks at 24.2% under prefix_on + chunked_on and bottoms out at 15.2% under prefix_on + chunked_off. Hard tasks (performance anti-patterns requiring query plan interpretation) are capped between 15% and 24% regardless of configuration. This hard-tier ceiling is the clearest evidence that the primary constraint on this system is model capability: Llama-3.1-8B does not reliably interpret multi-join execution plans and index-miss patterns within five reflection rounds, and no engine setting moves that number.

Full verbosity shows a directionally consistent but statistically insignificant advantage over compact across all tiers. Overall success is 39% (full) versus 35% (compact) (Fisher's exact $p = 0.66$). At the tier level, the gap is +6 pp on medium (39.4% vs 33.3%) and +6 pp on hard (24.2% vs 18.2%), while easy remains locked at 52.9% in both conditions. The direction is consistent with the hypothesis that compact truncation strips diagnostic detail: error messages are short and survive truncation intact, but long execution plans and multi-row result tables are cut at 600 characters, which is exactly the feedback medium- and hard-tier tasks depend on. The effect size is small ($r = 0.04$ on tokens, $r = 0.06$ on rounds) and no metric approaches significance (all $p > 0.34$).

Notably, compact prompts did not reduce token consumption: median tokens per task were 4,302 (compact) versus 4,210 (full), a 2% increase in the wrong direction ($U = 4792$, $p = 0.61$). The mechanism is task exhaustion: compact prompts lowered success, so more tasks ran all the way to the five-round budget and produced the maximum total token count. Token-saving measures that reduce signal are self-defeating at this model scale; the only effective way to reduce token consumption on failed tasks is to detect futility early and exit, as discussed in the Optimizations section.

6. Optimizations

Our measurements point to four optimization opportunities for a production deployment of a reflective agent, in decreasing order of confidence.

1. Prefix caching is necessary but incompletely exploited (measured). The 48% TTFT reduction came from caching the static per-task prefix. In a deployed multi-tenant system, the cacheable surface is larger: the system prompt and tool descriptions are shared across *all* tasks, and schema definitions are shared across all tasks targeting the same database. A schema-aware router that assigns tasks for the same database to the same engine replica would raise the cross-request cache hit rate well above what our single-task-stream benchmark achieves. The complementary lever is a prompt layout: ordering context from most-shared to least-shared (system prompt → tool spec → schema → task → history) maximizes the reusable prefix, which our agent already does by construction.

2. Cross-round KV reuse beyond the prefix (conceptual). Prefix caching only reuses the *unchanged leading* portion of the prompt. In reflection workloads, rounds n and n+1 share nearly the entire round-n prompt — the new content is appended, not interleaved. Engine-side support for append-only conversations (retaining the full per-task KV state between rounds rather than re-matching the radix tree [6] from scratch) would reduce round-n prefill to only the newly appended tokens, turning our observed 1.8× round-5 latency growth into near-constant per-round latency. This is the single largest remaining systems win for the reflection pattern, since per-round prompt growth is intrinsic to it.

3. Right-size the reflection budget by difficulty tier (measured). Success by round is tier-dependent: easy successes arrive at a median of 3 rounds versus 4 for medium and hard, and

hard tasks rarely succeed at any budget (15-24%). A deployed system should not spend a uniform 5-round budget; an early-exit policy (stop when the same error repeats or attempts to stop changing materially) and a tier-conditioned budget would cut the largest cost item, failed tasks that burn all 5 rounds, and the maximum token count (median ~4,500 tokens per failed task versus ~2,100-2,700 for successes). Our verbosity result reinforces this: compact prompts increased median tokens per task (4,302 vs 4,210) because they lowered success and pushed more tasks to budget exhaustion. Token-saving measures that reduce signal are counterproductive; budget-saving measures that detect futility are not.

4. Concurrency scaling and chunked prefill (conceptual). At concurrency 2 the engine is heavily underutilized (44× KV-cache headroom), and chunked prefill has nothing to do. A deployment would batch many agent tasks per replica, which raises throughput but reintroduces exactly the head-of-line risk chunked prefill addresses. Our null result should not be read as a license to disable it at scale. The interesting deployment question is the throughput/latency trade-off curve as concurrency rises toward the KV limit, with chunked prefill expected to flatten the tail [7]; measuring that curve at concurrency 8–32 is the experiment our setup was one knob away from running. If prefill-decode interference persists at that scale, disaggregating the two phases onto separate replicas, as in DistServe [10], is the heavier-weight fix.

One more point about where we think optimization effort should go. No engine configuration moved easy-tier success (see the 52.9% everywhere) or hard-tier success materially. Infrastructure optimizations in this setup brought latency and cost, not accuracy. Closing the hard tier requires a stronger model or richer tools (for example, loading row data so the agent can verify results), not a faster engine.

7. Related Work

Our agent is an instance of the reflection / self-correction pattern. Reflexion [1] and Self-Refine [2] established that models improve when they critique and retry their own outputs; we swap the model's self-critique for exact execution feedback from PostgreSQL, which is the strongest form of that signal available for SQL. Spider [5] supplies the underlying queries. Text-to-SQL generation is well studied; our contribution is not generation quality but a characterization of what the debugging loop costs the serving layer.

On the systems side we deliberately built on existing engine mechanisms rather than inventing new ones. vLLM [3] provides PagedAttention KV management and continuous batching in the lineage of Orca [9]. The two knobs we swept are RadixAttention prefix caching, introduced in SGLang [6], and chunked prefill, introduced by Sarathi-Serve [7]; DistServe [10] attacks the same prefill-decode interference by disaggregating the two phases onto separate hardware. Our experiment measures what these mechanisms, as productionized in vLLM on TPU, buy a reflection workload specifically.

8. Conclusion

We built a reflective SQL debugging agent and evaluated it across 100 Spider tasks under a two-way sweep of prompt verbosity and vLLM engine configuration. The reflection loop increased the success rate monotonically from 0% to 33–40% as the budget increased from 1 to 5 rounds. Prefix caching reduced mean TTFT by 48%. Chunked prefill showed no measurable effect. Prompt verbosity had no significant impact on success rate. The primary ceiling on hard tasks was model capability rather than infrastructure: Llama-3.1-8B could not reliably interpret multi-join schemas and query plans within 5 reflection rounds. Our code, the 100-task dataset, and all raw results are submitted alongside this writeup.

References

[1] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023. arXiv:2303.11366.
[2] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023. arXiv:2303.17651.
[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180.
[4] A. Grattafiori et al. The Llama 3 Herd of Models. arXiv:2407.21783, 2024.
[5] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. EMNLP 2018. arXiv:1809.08887.
[6] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104, 2023.
[7] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI 2024. arXiv:2403.02310.
[8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022. arXiv:2205.14135.
[9] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI 2022.
[10] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. OSDI 2024. arXiv:2401.09670.